A Self-Driving Car Simulator that Learns to Drive with Deep Q-Learning

A from-scratch NumPy implementation with a simple Python interface, inspired by Andrej Karpathy's self-driving-car demo

Vishwas N.
M.Tech in Digital Electronics and Communication
MSRIT (M.S. Ramaiah Institute of Technology)

November 1, 2026

Abstract

We build a complete, self-contained simulation of a self-driving car in plain Python. A car with nine range sensors ("eyes") must learn, purely from trial and error, to drive around a closed race track without touching the walls and as fast as possible. The learning agent is a Deep Q-Network (DQN) with experience replay, a target network, Double-DQN targets and a Huber loss. The neural network, its back-propagation and the Adam optimiser are all written by hand in NumPy (verified by numerical gradient checking); no deep-learning framework is used. A Tkinter interface lets the user watch the car learn in real time, change the simulation speed, save/load models and take over the steering with the keyboard. In experiments with three random seeds, the agent goes from crashing within a fraction of a lap to driving at the maximum allowed speed without crashing after about 1.7×10^4 training steps (under ten seconds of CPU time; the complete 150 000-step run takes about a minute on one core). The learned policy transfers, without retraining, to four randomly generated tracks it has never seen, where it never crashed in any evaluation episode. We also discuss what these results do and do not show.

Contents

1 Introduction **3**

2 Background **3**
2.1 Reinforcement learning and Q-learning . . . 3
2.2 Deep Q-Networks . . . 3

3 System Design **4**
3.1 Track generation . . . 4
3.2 Vehicle model . . . 4
3.3 Sensors . . . 5
3.4 Observation and reward . . . 5
3.5 Neural network and back-propagation . . . 6
3.6 The DQN training algorithm . . . 6

4 Software and User Interface **7**

5 Experiments **7**
5.1 Protocol . . . 7
5.2 Learning behaviour . . . 8

5.3 Quantitative results 8
5.4 Interpretation 10

6 Limitations and Honest Caveats **10**

7 Conclusion and Future Work **10**

A Complete Source Code **12**

1 Introduction

Teaching a vehicle to drive is a classic problem for reinforcement learning (RL): there is no dataset of "correct" steering commands, only a reward signal that tells the agent how well it is doing. A well-known and very approachable demonstration of this idea is Andrej Karpathy's browser-based *ConvNetJS* deep Q-learning demo [1], in which a small car with distance sensors learns to drive in traffic. Karpathy's other teaching projects, such as *micrograd* [2] and the *Hacker's Guide to Neural Networks* [3], share a philosophy that we adopt here: build every component yourself, keep it small enough to read in one sitting, and verify the gradients numerically.

Objectives.

1. Build a 2-D driving simulator with simple but sensible physics, sensors and a reward function.
2. Implement a Deep Q-Network completely from scratch (network, back-propagation, optimiser, replay, target network).
3. Provide a simple Python interface for watching the learning process live and for interacting with the car.
4. Evaluate the learned driver quantitatively against a random driver and a hand-written controller, including on tracks not seen during training.

Originality. This project is an *original re-implementation* in the spirit of the demos above; it does not reuse any of their code. The track generator, vehicle model, sensor geometry, reward function, network, DQN variant and user interface were all designed and written for this project. All source code is reproduced in Appendix A.

2 Background

2.1 Reinforcement learning and Q-learning

An agent interacts with an environment in discrete time steps. At step t it observes a state s_t, chooses an action a_t, receives a reward r_t and moves to a new state s_{t+1}. The goal is to maximise the expected discounted return $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$ with discount factor $\gamma \in [0, 1)$ [4]. The action-value function $Q^*(s, a)$ is the best achievable return after taking action a in state s; it satisfies the Bellman optimality equation

$$Q^*(s, a) = \mathbb{E}\left[r + \gamma \max_{a'} Q^*(s', a') \right], \tag{1}$$

and *Q-learning* [5] repeatedly moves $Q(s, a)$ towards the right-hand side. The greedy policy $\pi(s) = \arg\max_a Q(s, a)$ is then optimal.

2.2 Deep Q-Networks

When the state is a vector of real numbers, Q cannot be stored in a table and is approximated by a neural network $Q_\theta(s, \cdot)$, which outputs one value per action. Naively training such a network on consecutive experiences is unstable, so DQN [7] introduces two key ideas that we use:

- **Experience replay** [6]: transitions (s, a, r, s', d) are stored in a buffer and training uses random mini-batches, which breaks the correlation between consecutive samples.
- **Target network**: the regression target is computed with a slowly updated copy θ^- of the network, which keeps the target from chasing itself.

We additionally use **Double DQN** [8], which reduces over-estimation by letting the online network *choose* the next action and the target network *evaluate* it:

$$y = r + \gamma\,(1-d)\;Q_{\theta^-}\!\big(s',\ \arg\max_{a'} Q_\theta(s',a')\big). \tag{2}$$

The loss is the Huber loss of the temporal-difference (TD) error $\delta = Q_\theta(s,a) - y$ [9],

$$\ell(\delta) = \begin{cases} \tfrac{1}{2}\delta^2 & |\delta| \le 1 \\ |\delta| - \tfrac{1}{2} & \text{otherwise,} \end{cases} \qquad \frac{\partial \ell}{\partial \delta} = \operatorname{clip}(\delta, -1, 1), \tag{3}$$

which is quadratic for small errors and linear for large ones, so rare large errors (such as the crash penalty) do not destabilise training.

3 System Design

Figure 1 shows the overall learning loop. The environment produces an observation and a reward; the Q-network turns the observation into five action values; an ε-greedy rule picks the action; the transition is stored in the replay buffer; and every second step a mini-batch is used to improve the network.

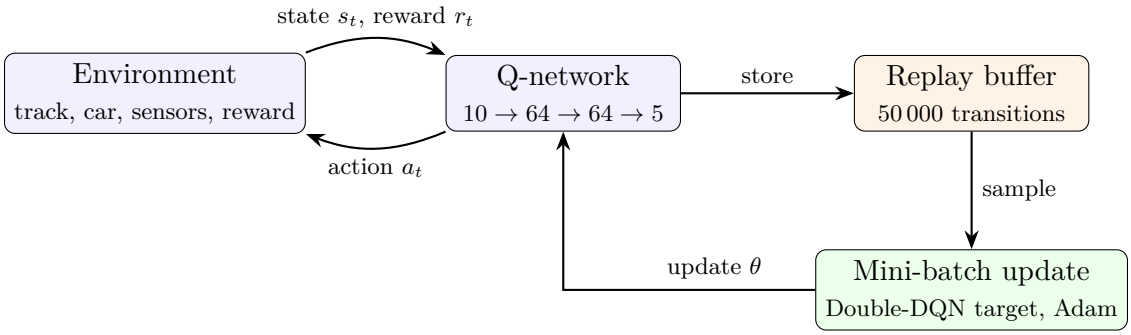


Figure 1: The learning loop. The environment and the Q-network exchange states, rewards and actions; every transition is stored in the replay buffer, and mini-batches sampled from it are used to update the parameters θ of the Q-network that chooses the actions.

3.1 Track generation

Each track is a closed loop whose centre line is a radial Fourier curve

$$r(\phi) = R_0 + \sum_{k=2}^{4} \frac{W}{k}\, u_k \cos(k\phi + \varphi_k), \qquad \phi \in [0, 2\pi), \tag{4}$$

with $R_0 = 250$, $W = 70$, and random $u_k \sim \mathcal{U}(0,1)$, $\varphi_k \sim \mathcal{U}(0, 2\pi)$ drawn from a seed. The centre line is sampled at $n = 360$ points. The signed curvature $\kappa = (x'y'' - y'x'')/(x'^2 + y'^2)^{3/2}$ is computed numerically, and a candidate curve is rejected unless its tightest bend has radius at least $2.3\,h$, where $h = 32$ is the road half-width. This guarantees that every generated track is physically drivable. Walls are the centre line offset by $\pm h$ along the unit normals, giving $2n$ wall segments. A different seed gives a different track, which we use to test generalisation.

3.2 Vehicle model

The car has position (x, y), heading θ and speed $v \in [v_{\min}, v_{\max}] = [1, 5]$ (units per step). There are five discrete actions: *coast*, *steer left* ($\theta \mathrel{+}= 0.07$ rad), *steer right* ($\theta \mathrel{-}= 0.07$ rad), *accelerate* ($v \mathrel{+}= 0.15$) and *brake* ($v \mathrel{-}= 0.30$). After applying the action the car moves by

$$x \leftarrow x + v\cos\theta, \qquad y \leftarrow y + v\sin\theta. \tag{5}$$

The steering rate is independent of speed, a deliberate simplification that keeps the dynamics easy to reason about. The car crashes (and the episode ends) when its distance to the nearest centre-line point exceeds $h - r_c$, with car radius $r_c = 6$. Progress is measured by the signed number of centre-line points the car has advanced, tracked with a windowed nearest-point search (so a car cannot "teleport" across to a distant part of the loop).

3.3 Sensors

The car is equipped with nine range sensors spread uniformly over a 180° fan centred on the heading (Figure 2). Sensor j casts a ray with direction $\mathbf{d}_j = (\cos(\theta + \alpha_j), \sin(\theta + \alpha_j))$ from the car position $\mathbf{o}$ and has maximum range $L = 120$. Against a wall segment starting at $\mathbf{p}$ with edge vector $\mathbf{e}$, the hit distance t and segment parameter u satisfy $\mathbf{o} + t\mathbf{d} = \mathbf{p} + u\mathbf{e}$, which gives

$$t = \frac{(\mathbf{p} - \mathbf{o}) \times \mathbf{e}}{\mathbf{d} \times \mathbf{e}}, \qquad u = \frac{(\mathbf{p} - \mathbf{o}) \times \mathbf{d}}{\mathbf{d} \times \mathbf{e}}, \qquad \mathbf{a} \times \mathbf{b} = a_x b_y - a_y b_x, \tag{6}$$

and a hit is valid when $t \geq 0$ and $0 \leq u \leq 1$. The distance returned by sensor j is the minimum valid t, or L if nothing is hit. The computation is fully vectorised over all rays and over the 202 wall segments nearest to the car, which makes it fast enough for thousands of simulation steps per second in pure NumPy.

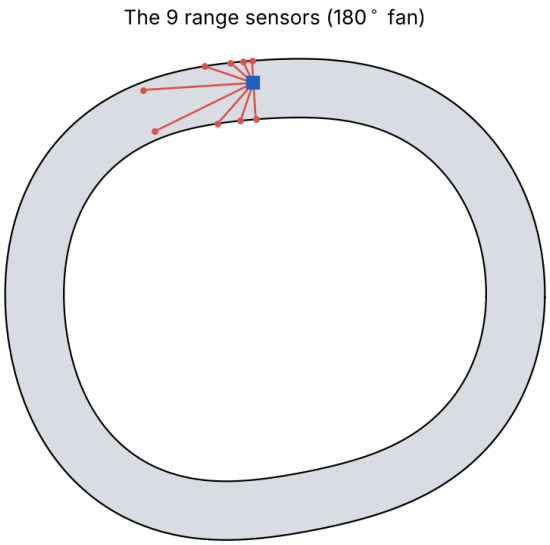


Figure 2: The nine range sensors of the car (blue square) on a generated track. Each ray reports the distance to the nearest wall, capped at 120 units.

3.4 Observation and reward

The observation is a 10-dimensional vector in $[0, 1]^{10}$: the nine ray distances divided by L, and the speed divided by $v_{\max}$. The agent does not receive its position, the map, or the road direction; it must infer everything from the sensors.

The reward at each step is

$$r = \frac{v}{v_{\max}} \cos(\theta - \theta_{\text{road}}) - \tfrac{1}{2}\, p^2, \qquad p = \max\left(0, \frac{0.25 - \min_j \hat{d}_j}{0.25}\right), \tag{7}$$

where θ_{road} is the direction of the road at the car's position and $\hat{d}_j$ are the normalised ray distances. The first term rewards fast driving *along* the road; the second is a mild penalty when

any sensor sees a wall closer than a quarter of its range, which encourages staying away from the walls. A crash gives $r = -1$ and ends the episode. Episodes also end after a fixed step limit (2000 in training); such time-limit endings are *not* treated as terminal in the Bellman target, since the car is not actually in a terminal state.

3.5 Neural network and back-propagation

The Q-network is a multilayer perceptron $10 \to 64 \to 64 \to 5$ with ReLU hidden units and a linear output (5 189 parameters). Weights use He initialisation [10]. For a mini-batch X the forward pass is $h_0 = X$, $h_i = \mathrm{ReLU}(h_{i-1}W_i + b_i)$ for hidden layers, and a linear last layer. Given $\partial \ell / \partial h_L$ (non-zero only for the action that was taken), the backward pass is

$$\frac{\partial \ell}{\partial W_i} = h_{i-1}^{\top} \Delta_i, \qquad \frac{\partial \ell}{\partial b_i} = \sum_{\text{batch}} \Delta_i, \qquad \Delta_{i-1} = (\Delta_i W_i^{\top}) \odot \mathbb{1}[h_{i-1} > 0]. \tag{8}$$

Parameters are updated with Adam [11] ($\beta_1 = 0.9$, $\beta_2 = 0.999$) with global gradient-norm clipping at 10.

Gradient check. As in the micrograd tradition, the hand-written gradients are verified against centred finite differences, $(\mathcal{L}(w + \epsilon) - \mathcal{L}(w - \epsilon))/2\epsilon$ with $\epsilon = 10^{-6}$, on a random $6 \to 8 \to 7 \to 4$ network. The worst relative error over all weights and biases is 3.1×10^{-7}, well below the 10^{-5} threshold, so the back-propagation is correct.

3.6 The DQN training algorithm

1. Initialise the online network Q_θ and copy it to the target network Q_{θ^-}; empty the replay buffer.
2. At each environment step, choose a random action with probability ε (decayed linearly from 1.0 to 0.05 over 25 000 steps), otherwise $a = \arg\max_a Q_\theta(s, a)$.
3. Execute a, observe r, s' and the crash flag d; store (s, a, r, s', d) in the replay buffer.
4. After a warm-up of 1 000 steps, every second step sample a mini-batch of 64 transitions, compute the targets by Eq. (2), take one Adam step on the Huber loss.
5. Every 500 updates copy θ into θ^-.
6. When an episode ends (crash or step limit), reset the car to a random point on the track with a random lateral offset, heading noise and speed.

Random start positions matter: without them the agent would only ever practise the first corner of the track.

Table 1: Hyper-parameters used in all experiments (no tuning was performed).

Learning		**Environment**	
Discount γ	0.97	Road half-width h	32
Learning rate (Adam)	10^{-3}	Sensor range L / count	120 / 9
Batch size	64	Speed range $[v_{\min}, v_{\max}]$	[1, 5]
Replay capacity	50 000	Steering per step	0.07 rad
Target sync	every 500 updates	Acceleration / braking	+0.15 / −0.30
Update frequency	every 2 steps	Car radius r_c	6
Warm-up	1 000 steps	Episode limit (train / eval)	2 000 / 3 000
ε schedule	$1 \to 0.05$ in 25 000 steps	Hidden layers	64, 64 (ReLU)

4 Software and User Interface

All code is in a single file, `self_driving_car.py`, written in Python with NumPy; the interface uses Tkinter, which is part of the standard Python distribution. The file is organised in clearly separated components:

Component	Responsibility
`Track`	track generation, wall geometry, nearest-point search
`CarEnv`	vehicle dynamics, ray casting, reward, `reset()/step()` interface
`MLP`	network forward/backward pass, Adam, save/load, gradient check
`DQNAgent`	replay buffer, Double-DQN update, target network
`Trainer`	exploration schedule, episode bookkeeping and history
`App`	Tkinter interface
CLI	`gui`, `train`, `eval`, `gradcheck` commands

The interface. Running `python self_driving_car.py` opens a window with the track on the left and a control panel on the right. The car and its nine sensor rays are drawn live (green/yellow/red according to how close the wall is). The panel offers:

- **Start/Pause** training (also bound to the space bar) and a slider for the number of simulation steps per displayed frame (1–400), so learning can be watched in slow motion or fast-forwarded;
- **Reset car**, **New random track** (the agent keeps its knowledge, which demonstrates generalisation live), and **Reset agent**;
- **Drive manually**: the user takes over with the arrow keys, which is useful to appreciate how difficult the task is;
- **Greedy** mode (no random exploration), **Show sensors** toggle, and **Save/Load model**;
- live statistics (steps, episodes, ε, average distance, return and crash rate over the last 50 episodes, TD loss) and a small learning curve.

The quantitative results in this report were produced with the headless commands (`train`, `eval`), which use exactly the same `Trainer` and environment code as the interface.

5 Experiments

5.1 Protocol

The agent is trained for 150 000 environment steps on a single track (seed 1) and is then evaluated with the greedy policy ($\varepsilon = 0$). Each evaluation consists of 20 episodes of up to 3 000 steps, starting at random points on the track. We evaluate on the training track and on four unseen tracks (seeds 2–5) generated by the same procedure. Three policies are compared:

Random: uniformly random actions.

Heuristic: a hand-written controller that steers toward the side with more free space, brakes if something is straight ahead, and otherwise accelerates up to about 80 % of the top speed.

DQN: the trained agent.

We report (i) *mean laps*: the distance travelled along the track before crashing or reaching the 3 000-step limit, expressed in laps; (ii) the *survival rate*: the fraction of episodes that did not end in a crash; and (iii) the mean speed. The whole training procedure was repeated with three random seeds (0, 1, 2), which change the network initialisation, exploration and start-position randomness, but not the training track.

5.2 Learning behaviour

Figure 3 shows the learning curve of seed 0. For the first roughly 10 000 steps the car crashes after a short distance. Between about 10 000 and 17 000 steps, the performance jumps: the first episode that lasts the full 2 000 steps without crashing occurs at step 17 931 (seed 0), 17 036 (seed 1) and 16 909 (seed 2), i.e. $(1.73 \pm 0.05) \times 10^4$ steps; this happens while ε is still above 0.3, i.e. the car already survives 2 000 steps even though roughly a third of its actions are random. Because episodes become much longer once the car survives, only 167–192 episodes are completed in the whole run, of which 101–126 ended in a crash. Of the 195 episodes that ended after step 20 000 (65 per seed), only 2 were crashes (one in seed 0 and one in seed 1). The curve is nearly flat afterwards, with one visible dip near step 81 000 (seed 0) where a single episode ended early with a crash; this is a typical sign of the residual instability of value-based deep RL, and the policy recovered immediately.

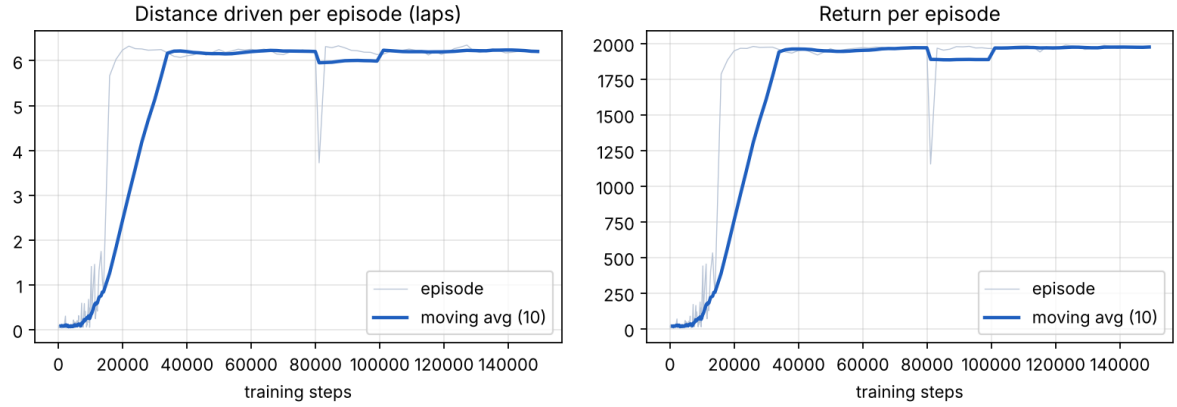


Figure 3: Learning curve (seed 0). The light line shows individual episodes, each plotted at the training step where it ended; the thick line is a 10-episode moving average. Episodes are capped at 2 000 steps, so distance saturates near 6.3 laps.

5.3 Quantitative results

Table 2 gives the per-track results for seed 0 and Table 3 summarises the three seeds.

Table 2: Mean laps over 20 evaluation episodes (3 000-step limit) per track, for the seed-0 agent. Survival rate was 0.00 for the random driver and 1.00 for both the heuristic and the DQN on every track.

Track	Random	Heuristic	DQN	DQN std. dev.
Training track (seed 1)	0.08	7.88	9.19	0.01
Unseen track (seed 2)	0.08	7.79	9.06	0.01
Unseen track (seed 3)	0.08	7.84	9.14	0.01
Unseen track (seed 4)	0.08	7.63	8.89	0.01
Unseen track (seed 5)	0.09	7.80	9.10	0.01

Table 3: Results for three independent training runs. "Unseen" is the average over the four unseen tracks. Survival of the DQN was 1.00 in every case.

Seed	First crash-free full episode (step)	DQN laps, training track	DQN laps, unseen (mean)	Training time (s)
0	17 931	9.19	9.05	54
1	17 036	9.48	9.36	57
2	16 909	9.30	9.20	56
Mean ± std	$17\,292 \pm 455$	9.32 ± 0.12	9.20 ± 0.13	56

For reference (identical for every seed, since they are deterministic): heuristic 7.76 laps on the unseen tracks, random driver 0.08 laps.

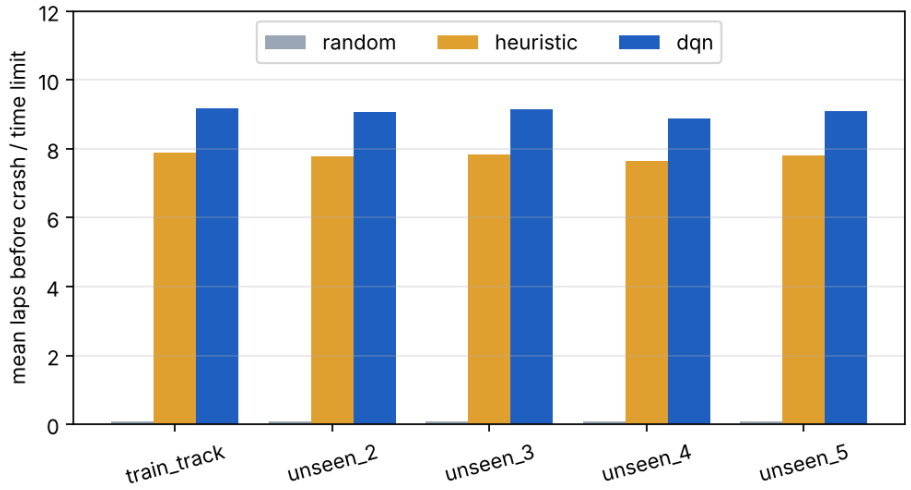


Figure 4: Mean laps for the three policies on the training track and four unseen tracks (seed-0 agent).

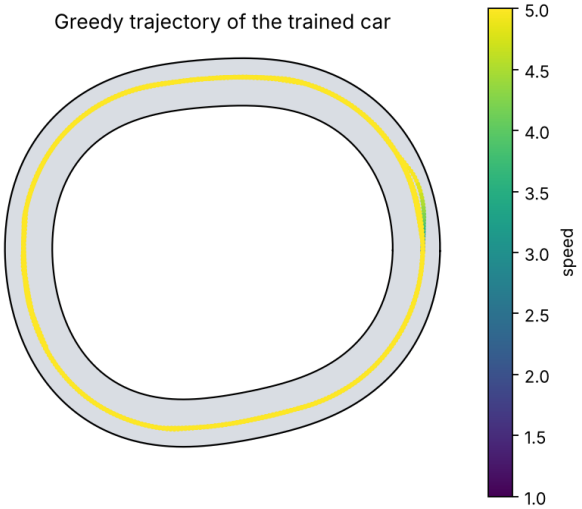


Figure 5: Greedy trajectory of the trained car on the training track, coloured by speed. After a short initial correction the car holds the maximum speed (5) and follows the middle of the road.

5.4 Interpretation

- **Learning works.** The random driver covers 0.08 laps before crashing; the trained agent never crashed in any of the 5×20 evaluation episodes per seed, i.e. it safely drove for 3 000 steps each time.
- **Why the DQN scores more laps than the heuristic.** Both controllers survive every episode, so the lap count is mainly a measure of *speed*. The DQN drives at an average speed of 4.99 (essentially the top speed 5), whereas the heuristic averages 4.07. With a 3 000-step limit and a track length of about 1 583 units, the theoretical maximum is $3000 \times 5/1583 \approx 9.5$ laps, and the DQN reaches 8.9–9.5 across all seeds and tracks. The comparison should therefore be read as "the learned agent drives as fast as physically allowed without crashing, while the simple controller is cautious", not as evidence that the DQN is a better steerer in general. The heuristic is a deliberately simple baseline and was not tuned for speed.
- **Generalisation.** Without any retraining, the agent drove all four unseen tracks without a crash and with almost the same distance as on the training track (Table 3). This is plausible because the observation consists only of *relative* quantities (distances to walls and speed), so a good policy for one bend is also a good policy for similar bends elsewhere.
- **Cost.** Training took about 56 seconds on one CPU core (about 2 700 steps/s including the gradient updates), which is why a pure-NumPy implementation is perfectly adequate for this problem.

6 Limitations and Honest Caveats

- **Same family of tracks.** The unseen tracks come from the same random generator as the training track (smooth loops of similar width and curvature). Success on them does not imply that the agent could handle sharp hairpins, intersections, or narrower roads.
- **Saturated metric.** Because both good controllers survive the whole evaluation, the lap metric is capped by the speed limit and the 3 000-step window. A finer comparison would require lap times, harder tracks, or longer horizons.
- **Simplified world.** The vehicle model has no tyre slip, inertia or speed-dependent steering, and there is no other traffic. The state is only the current sensor reading plus speed (no memory), so the agent cannot, for example, anticipate a bend it cannot yet see.
- **No hyper-parameter search; few seeds.** A single hyper-parameter setting was used and only three seeds were run. The results are consistent across these seeds, but three seeds are not enough for strong statistical claims, and the small standard deviations in Table 3 should be read in that light. The single dip in Figure 3 shows that training is not perfectly stable.
- **Interface vs. experiments.** The graphical interface is a thin layer over the same trainer and environment classes, but the numbers in this report were produced with the headless commands, not through the window.

7 Conclusion and Future Work

We implemented, from scratch and in plain Python/NumPy, a complete self-driving-car learning system: a procedurally generated track, a sensor-equipped car, a hand-written neural network with verified gradients, a Double-DQN agent, and an interactive interface. The agent learns to drive crash-free at maximum speed in roughly 17 000 steps (under ten seconds of CPU time;

the full 150 000-step run takes under a minute) and transfers to unseen tracks of the same kind without retraining.

Natural extensions are: (i) pixel observations with a convolutional network, as in the original ConvNetJS demonstrations; (ii) other cars and traffic, which makes collision avoidance and overtaking part of the task; (iii) continuous steering and throttle with an actor–critic method; (iv) more demanding track families (hairpins, varying width) and a curriculum; (v) a recurrent or frame-stacked state so the agent has memory; and (vi) prioritised replay or n-step returns to speed up learning further.

How to Reproduce

```
pip install numpy matplotlib          # matplotlib is optional (figures)
python self_driving_car.py gradcheck  # verify back-propagation
python self_driving_car.py            # open the interactive window
python self_driving_car.py train --steps 150000 --out runs/run1 --seed 0
python self_driving_car.py eval --model runs/run1/model.npz
```

The `train` command writes the model, the learning history, a JSON file of evaluation results and the figures used in this report (`learning_curve`, `trajectory`, `sensors`, `comparison`) into the output folder. Seeds 1 and 2 correspond to `-seed 1` and `-seed 2`. Absolute timings depend on the machine; the numerical results depend on the NumPy version only through floating-point details.

References

[1] A. Karpathy. *ConvNetJS: Deep Q Learning demo* (browser-based self-driving car with distance sensors). https://cs.stanford.edu/people/karpathy/convnetjs/demo/rldemo.html.

[2] A. Karpathy. *micrograd*: a tiny scalar-valued autograd engine. https://github.com/karpathy/micrograd.

[3] A. Karpathy. *Hacker's guide to Neural Networks*. https://karpathy.github.io/neuralnets/.

[4] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[5] C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.

[6] L.-J. Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8:293–321, 1992.

[7] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[8] H. van Hasselt, A. Guez and D. Silver. Deep reinforcement learning with double Q-learning. In *AAAI*, 2016.

[9] P. J. Huber. Robust estimation of a location parameter. *Annals of Mathematical Statistics*, 35(1):73–101, 1964.

[10] K. He, X. Zhang, S. Ren and J. Sun. Delving deep into rectifiers: surpassing human-level performance on ImageNet classification. In *ICCV*, 2015.

[11] D. P. Kingma and J. Ba. Adam: a method for stochastic optimization. In *ICLR*, 2015.

A Complete Source Code

The listing below is the complete file `self_driving_car.py`.

```
#!/usr/bin/env python3
"""
self_driving_car.py  --  A self-driving car that learns to drive by itself.
=============================================================================

A from-scratch re-creation (in spirit) of Andrej Karpathy's well-known
"ConvNetJS Deep Q-Learning: Self-Driving Car" demo, written in plain Python
with NumPy only (the GUI uses Tkinter, which ships with Python).

What is in this single file
---------------------------
  Track      : procedurally generated closed race track (random smooth loop).
  CarEnv     : simple 2-D car physics + 9 distance sensors ("eyes") + reward.
  MLP        : a small neural network with hand-written back-propagation and
               the Adam optimiser (no PyTorch / TensorFlow).
  DQNAgent   : Deep Q-Network with experience replay, a target network,
               Double-DQN targets and a Huber loss.
  Trainer    : glue code - epsilon-greedy exploration, episode bookkeeping.
  App        : Tkinter interface (train live, watch the car, drive manually).
  CLI        : headless training / evaluation / gradient check.

Inspired by Karpathy's projects (re-implemented, not copied):
  * ConvNetJS "REINFORCEjs / Deep Q Learning self-driving car"  -> the task,
    the "eyes" (range sensors), the reward idea (go fast, stay off walls).
  * micrograd / "Hacker's guide to Neural Networks" -> a tiny network that is
    trained with hand-written gradients and verified by numerical
    gradient checking (see `python self_driving_car.py gradcheck`).

Usage
-----
  python self_driving_car.py                        # open the GUI (default)
  python self_driving_car.py gui --model m.npz      # GUI, start from a model
  python self_driving_car.py train --steps 150000 --out runs/run1
  python self_driving_car.py eval  --model runs/run1/model.npz
  python self_driving_car.py gradcheck              # verify back-propagation

Requirements: Python 3.9+, numpy.  Optional: matplotlib (figures), tkinter (GUI).
"""
from __future__ import annotations

import argparse
import json
import math
import os
import sys
import time

import numpy as np

# ============================================================================
# 1.  TRACK
# ============================================================================


def wrap_angle(a: float) -> float:
    """Wrap an angle to (-pi, pi]."""
    return (a + math.pi) % (2.0 * math.pi) - math.pi


class Track:
    """A closed race track described by a smooth centre-line and two walls.

    The centre line is a radial Fourier curve
        r(t) = R0 + sum_{k=2..4} a_k cos(k t + phi_k),   t in [0, 2 pi)
    Random amplitudes/phases (from `seed`) give a different track each time.
    Curves whose curvature is too high for the road width are rejected, so
    every generated track is drivable.
    """

```

```
    def __init__(self, seed: int = 1, n: int = 360, base_radius: float = 250.0,
                 half_width: float = 32.0, wiggle: float = 70.0):
        self.seed, self.n, self.hw = int(seed), int(n), float(half_width)
        rng = np.random.default_rng(self.seed)
        t = np.linspace(0.0, 2.0 * np.pi, self.n, endpoint=False)
        dt = 2.0 * np.pi / self.n
        for _ in range(1000):
            r = np.full_like(t, base_radius)
            for k in (2, 3, 4):
                r += rng.uniform(0.0, 1.0) * wiggle / k * np.cos(k * t + rng.uniform(0, 2
                    * np.pi))
            x, y = r * np.cos(t), r * np.sin(t)
            xp = (np.roll(x, -1) - np.roll(x, 1)) / (2 * dt)
            yp = (np.roll(y, -1) - np.roll(y, 1)) / (2 * dt)
            xpp = (np.roll(x, -1) - 2 * x + np.roll(x, 1)) / dt ** 2
            ypp = (np.roll(y, -1) - 2 * y + np.roll(y, 1)) / dt ** 2
            kappa = (xp * ypp - yp * xpp) / np.maximum((xp ** 2 + yp ** 2) ** 1.5, 1e-9)
            if np.abs(kappa).max() < 1.0 / (2.3 * self.hw):   # min turn radius > 2.3 *
                half-width
                break
        self.C = np.stack([x, y], axis=1)                      # centre line (n, 2)
        tang = np.stack([xp, yp], axis=1)
        tang /= np.linalg.norm(tang, axis=1, keepdims=True)
        self.T = tang                                          # unit tangents
        self.Nrm = np.stack([-tang[:, 1], tang[:, 0]], axis=1)  # left normals (point
            inwards, CCW loop)
        self.tang_angle = np.arctan2(tang[:, 1], tang[:, 0])
        self.inner = self.C + self.hw * self.Nrm
        self.outer = self.C - self.hw * self.Nrm
        # wall segments, shape (2 walls, n, 2): start points P and edge vectors E
        P = np.stack([self.inner, self.outer])
        Q = np.roll(P, -1, axis=1)
        self.P, self.E = P, Q - P
        self.length = float(np.linalg.norm(np.roll(self.C, -1, 0) - self.C, axis=1).sum()
            )
        self._win = np.arange(-25, 26)         # window for fast nearest-point search
        self._seg_win = np.arange(-50, 51)     # window of wall segments used for ray
            casting

    def nearest(self, x: float, y: float, hint=None) -> int:
        """Index of the centre-line point closest to (x, y)."""
        if hint is None:
            d = (self.C[:, 0] - x) ** 2 + (self.C[:, 1] - y) ** 2
            return int(d.argmin())
        w = (hint + self._win) % self.n
        pts = self.C[w]
        d = (pts[:, 0] - x) ** 2 + (pts[:, 1] - y) ** 2
        return int(w[d.argmin()])

    def local_segments(self, idx: int):
        """Wall segments (start points, edge vectors) near centre-line index idx."""
        w = (idx + self._seg_win) % self.n
        return self.P[:, w].reshape(-1, 2), self.E[:, w].reshape(-1, 2)


# ============================================================================
# 2.  ENVIRONMENT  (car + sensors + reward)
# ============================================================================


class CarEnv:
    """2-D car on a Track.  Sensors are 9 distance 'eyes' spread over 180 degrees.

    Observation (10 numbers, all in [0, 1]):
        9 normalised ray distances (0 = touching a wall, 1 = nothing within range)
        1 normalised speed
    Actions (5):  0 coast | 1 steer left | 2 steer right | 3 accelerate | 4 brake
    Reward per step:
        + (speed / v_max) * cos(heading error w.r.t. the road direction)
        - 0.5 * proximity^2   (only when some sensor sees a wall closer than 25 %)
        -1 and the episode ends when the car touches a wall.
    """

```

```
    N_RAYS = 9
    N_ACTIONS = 5
    OBS_DIM = N_RAYS + 1
    ACTION_NAMES = ["coast", "left", "right", "accelerate", "brake"]

    def __init__(self, track: Track, ray_len: float = 120.0, v_min: float = 1.0,
                 v_max: float = 5.0, turn: float = 0.07, acc: float = 0.15,
                 brk: float = 0.30, car_r: float = 6.0, max_steps: int = 2000,
                 rng=None):
        self.track = track
        self.L, self.v_min, self.v_max = ray_len, v_min, v_max
        self.turn, self.acc, self.brk, self.car_r = turn, acc, brk, car_r
        self.max_steps = max_steps
        self.rng = rng if rng is not None else np.random.default_rng(0)
        self.ray_angles = np.linspace(-np.pi / 2, np.pi / 2, self.N_RAYS)
        self.ray_dists = np.full(self.N_RAYS, ray_len)
        self.ray_dirs = np.zeros((self.N_RAYS, 2))
        self.reset()

    # -- helpers ---------------------------------------------------------
    def set_track(self, track: Track):
        self.track = track
        return self.reset()

    def _cast_rays(self):
        """Vectorised ray / wall-segment intersection."""
        ang = self.th + self.ray_angles
        D = np.stack([np.cos(ang), np.sin(ang)], axis=1)                # (K, 2)
        P, E = self.track.local_segments(self.idx)                      # (M, 2) each
        wx, wy = P[:, 0] - self.x, P[:, 1] - self.y                     # (M,)
        denom = D[:, 0:1] * E[:, 1] - D[:, 1:2] * E[:, 0]               # d x e   (K, M)
        with np.errstate(divide="ignore", invalid="ignore"):
            t = (wx * E[:, 1] - wy * E[:, 0]) / denom                   # distance along
                ray
            u = (wx * D[:, 1:2] - wy * D[:, 0:1]) / denom               # position on
                segment
        ok = (np.abs(denom) > 1e-9) & (t >= 0) & (u >= 0) & (u <= 1)
        self.ray_dists = np.minimum(np.where(ok, t, self.L).min(axis=1), self.L)
        self.ray_dirs = D

    def _observe(self):
        self._cast_rays()
        return np.concatenate([self.ray_dists / self.L, [self.v / self.v_max]])

    # -- gym-like API ----------------------------------------------------
    def reset(self, start_idx=None, random_start: bool = True):
        tr = self.track
        if start_idx is None:
            start_idx = int(self.rng.integers(tr.n)) if random_start else 0
        i = int(start_idx) % tr.n
        off = self.rng.uniform(-0.3, 0.3) * tr.hw                       # lateral offset
        p = tr.C[i] + off * tr.Nrm[i]
        self.x, self.y = float(p[0]), float(p[1])
        self.th = float(tr.tang_angle[i] + self.rng.uniform(-0.1, 0.1))
        self.v = float(self.rng.uniform(1.5, 3.0))
        self.idx, self.start_idx = i, i
        self.progress = 0                  # signed number of centre-line points
            travelled
        self.steps = 0
        self.crashed = False
        return self._observe()

    def step(self, a: int):
        if a == 1:
            self.th += self.turn
        elif a == 2:
            self.th -= self.turn
        elif a == 3:
            self.v = min(self.v_max, self.v + self.acc)
        elif a == 4:
            self.v = max(self.v_min, self.v - self.brk)
        self.x += self.v * math.cos(self.th)
        self.y += self.v * math.sin(self.th)
```

```
        self.steps += 1

        tr = self.track
        idx = tr.nearest(self.x, self.y, self.idx)
        self.progress += ((idx - self.idx + tr.n // 2) % tr.n) - tr.n // 2
        self.idx = idx
        off_centre = math.hypot(self.x - tr.C[idx, 0], self.y - tr.C[idx, 1])
        self.crashed = off_centre > tr.hw - self.car_r

        obs = self._observe()
        if self.crashed:
            return obs, -1.0, True, {"crashed": True}
        err = wrap_angle(self.th - tr.tang_angle[idx])
        prox = max(0.0, (0.25 - obs[: self.N_RAYS].min()) / 0.25)
        reward = (self.v / self.v_max) * math.cos(err) - 0.5 * prox * prox
        return obs, reward, False, {"crashed": False}

    @property
    def laps(self) -> float:
        return self.progress / self.track.n


# ============================================================================
# 3.  NEURAL NETWORK WITH HAND-WRITTEN BACK-PROPAGATION
# ============================================================================


class MLP:
    """Fully connected network: ReLU hidden layers, linear output, Adam optimiser."""

    def __init__(self, sizes, rng=None, dtype=np.float64):
        rng = rng if rng is not None else np.random.default_rng(0)
        self.sizes = list(sizes)
        self.W = [rng.standard_normal((a, b)).astype(dtype) * math.sqrt(2.0 / a)   # He
            init
                  for a, b in zip(sizes[:-1], sizes[1:])]
        self.b = [np.zeros(b, dtype=dtype) for b in sizes[1:]]
        self.m = [np.zeros_like(p) for p in self.W + self.b]
        self.v = [np.zeros_like(p) for p in self.W + self.b]
        self.t = 0

    def forward(self, X):
        """Returns (output, list of layer activations) - activations are kept for
            backward()."""
        acts, h = [X], X
        last = len(self.W) - 1
        for i, (W, b) in enumerate(zip(self.W, self.b)):
            z = h @ W + b
            h = z if i == last else np.maximum(z, 0.0)
            acts.append(h)
        return h, acts

    def predict(self, X):
        return self.forward(X)[0]

    def backward(self, acts, dout):
        """Gradients of the loss w.r.t. all parameters, given dLoss/dOutput."""
        gW, gb = [None] * len(self.W), [None] * len(self.W)
        for i in reversed(range(len(self.W))):
            gW[i] = acts[i].T @ dout
            gb[i] = dout.sum(axis=0)
            if i > 0:
                dout = (dout @ self.W[i].T) * (acts[i] > 0)               # ReLU
                    derivative
        return gW, gb

    def adam_step(self, gW, gb, lr=1e-3, beta1=0.9, beta2=0.999, eps=1e-8, clip=10.0):
        grads = gW + gb
        norm = math.sqrt(sum(float((g ** 2).sum()) for g in grads))
        if norm > clip:                                                    # global-norm
             clipping
            grads = [g * (clip / norm) for g in grads]
        self.t += 1
```

```
        params = self.W + self.b
        for k, (p, g) in enumerate(zip(params, grads)):
            self.m[k] = beta1 * self.m[k] + (1 - beta1) * g
            self.v[k] = beta2 * self.v[k] + (1 - beta2) * g * g
            mh = self.m[k] / (1 - beta1 ** self.t)
            vh = self.v[k] / (1 - beta2 ** self.t)
            p -= lr * mh / (np.sqrt(vh) + eps)                                  # in-place
                update

    def copy_from(self, other: "MLP"):
        for a, b in zip(self.W + self.b, other.W + other.b):
            a[...] = b

    def save(self, path):
        d = {f"W{i}": w for i, w in enumerate(self.W)}
        d.update({f"b{i}": b for i, b in enumerate(self.b)})
        np.savez(path, sizes=np.array(self.sizes), **d)

    def load(self, path):
        z = np.load(path)
        assert list(z["sizes"]) == self.sizes, f"architecture mismatch: {z['sizes']} vs {
            self.sizes}"
        for i in range(len(self.W)):
            self.W[i][...] = z[f"W{i}"]
            self.b[i][...] = z[f"b{i}"]


def gradient_check(verbose: bool = True) -> float:
    """Compare analytic back-propagation with centred finite differences."""
    rng = np.random.default_rng(1)
    net = MLP([6, 8, 7, 4], rng)
    X, Y = rng.standard_normal((5, 6)), rng.standard_normal((5, 4))

    def loss():
        return 0.5 * float(((net.predict(X) - Y) ** 2).sum()) / len(X)

    out, acts = net.forward(X)
    gW, gb = net.backward(acts, (out - Y) / len(X))
    worst, h = 0.0, 1e-6
    for params, grads, name in ((net.W, gW, "W"), (net.b, gb, "b")):
        for li, (p, g) in enumerate(zip(params, grads)):
            for idx in np.ndindex(p.shape):
                old = p[idx]
                p[idx] = old + h
                lp = loss()
                p[idx] = old - h
                lm = loss()
                p[idx] = old
                num = (lp - lm) / (2 * h)
                rel = abs(num - g[idx]) / max(1e-8, abs(num) + abs(g[idx]))
                worst = max(worst, rel)
    if verbose:
        print(f"gradient check: worst relative error = {worst:.3e}  ({'PASS' if worst < 1
            e-5 else 'FAIL'})")
    return worst


# ============================================================================
# 4.  DEEP Q-NETWORK AGENT
# ============================================================================


class DQNAgent:
    """Deep Q-learning with replay buffer, target network and Double-DQN targets."""

    def __init__(self, obs_dim=CarEnv.OBS_DIM, n_actions=CarEnv.N_ACTIONS, hidden=(64,
        64),
                 gamma=0.97, lr=1e-3, batch=64, buffer=50_000, target_every=500, seed=0):
        self.rng = np.random.default_rng(seed)
        sizes = [obs_dim, *hidden, n_actions]
        self.q, self.q_target = MLP(sizes, self.rng), MLP(sizes, self.rng)
        self.q_target.copy_from(self.q)
        self.n_actions, self.gamma, self.lr = n_actions, gamma, lr
```

```
        self.batch, self.cap, self.target_every = batch, buffer, target_every
        self.S = np.zeros((buffer, obs_dim))
        self.S2 = np.zeros((buffer, obs_dim))
        self.A = np.zeros(buffer, dtype=np.int64)
        self.R = np.zeros(buffer)
        self.D = np.zeros(buffer)
        self.size = self.pos = self.n_updates = 0

    def act(self, obs, eps: float, rng=None) -> int:
        rng = rng if rng is not None else self.rng
        if eps > 0 and rng.random() < eps:
            return int(rng.integers(self.n_actions))
        return int(self.q.predict(obs[None, :])[0].argmax())

    def remember(self, s, a, r, s2, done):
        i = self.pos
        self.S[i], self.A[i], self.R[i], self.S2[i], self.D[i] = s, a, r, s2, float(done)
        self.pos = (i + 1) % self.cap
        self.size = min(self.size + 1, self.cap)

    def learn(self) -> float:
        idx = self.rng.integers(0, self.size, size=self.batch)
        S, A, R, S2, D = self.S[idx], self.A[idx], self.R[idx], self.S2[idx], self.D[idx]
        a_star = self.q.predict(S2).argmax(axis=1)                          # Double DQN:
             online net picks...
        q_next = self.q_target.predict(S2)[np.arange(self.batch), a_star]  # ...target
            net evaluates
        y = R + self.gamma * (1.0 - D) * q_next
        out, acts = self.q.forward(S)
        rows = np.arange(self.batch)
        td = out[rows, A] - y
        loss = float(np.where(np.abs(td) <= 1, 0.5 * td ** 2, np.abs(td) - 0.5).mean())
              # Huber
        dout = np.zeros_like(out)
        dout[rows, A] = np.clip(td, -1.0, 1.0) / self.batch                  # d Huber /
            d q
        gW, gb = self.q.backward(acts, dout)
        self.q.adam_step(gW, gb, lr=self.lr)
        self.n_updates += 1
        if self.n_updates % self.target_every == 0:
            self.q_target.copy_from(self.q)
        return loss

    def save(self, path):
        self.q.save(path)

    def load(self, path):
        self.q.load(path)
        self.q_target.copy_from(self.q)


# ============================================================================
# 5.  TRAINER  (exploration schedule, episode bookkeeping)
# ============================================================================


class Trainer:
    def __init__(self, track_seed=1, seed=0, eps_start=1.0, eps_end=0.05, eps_steps=25
        _000,
                 warmup=1_000, train_every=2, **agent_kw):
        self.rng = np.random.default_rng(seed + 1000)
        self.track_seed = track_seed
        self.env = CarEnv(Track(track_seed), rng=np.random.default_rng(seed + 2000))
        self.agent = DQNAgent(seed=seed, **agent_kw)
        self.eps_start, self.eps_end, self.eps_steps = eps_start, eps_end, eps_steps
        self.warmup, self.train_every = warmup, train_every
        self.total_steps = 0
        self.loss_ema = 0.0
        self.ep_ret = 0.0
        self.hist = {"step": [], "ret": [], "laps": [], "length": [], "crash": []}
        self.obs = self.env.reset()

    def epsilon(self) -> float:
```

```
        f = min(1.0, self.total_steps / self.eps_steps)
        return self.eps_start + f * (self.eps_end - self.eps_start)

    def resync(self):
        """Call after the environment was changed from outside (manual driving, new track)."""
        self.obs = self.env._observe()
        self.ep_ret = 0.0

    def set_track(self, seed):
        self.track_seed = seed
        self.obs = self.env.set_track(Track(seed))
        self.ep_ret = 0.0

    def reset_agent(self, seed=0):
        kw = dict(hidden=tuple(self.agent.q.sizes[1:-1]))
        self.agent = DQNAgent(seed=seed, **kw)
        self.total_steps, self.loss_ema = 0, 0.0
        self.hist = {k: [] for k in self.hist}
        self.resync()

    def step(self, greedy: bool = False):
        env, ag = self.env, self.agent
        eps = 0.0 if greedy else self.epsilon()
        a = ag.act(self.obs, eps, self.rng)
        obs2, r, crashed, _ = env.step(a)
        if not greedy:
            ag.remember(self.obs, a, r, obs2, crashed)       # time-limit endings are NOT terminal
            self.total_steps += 1
            if self.total_steps > self.warmup and self.total_steps % self.train_every == 0:
                self.loss_ema = 0.99 * self.loss_ema + 0.01 * ag.learn()
        self.ep_ret += r
        if crashed or env.steps >= env.max_steps:
            h = self.hist
            h["step"].append(self.total_steps)
            h["ret"].append(self.ep_ret)
            h["laps"].append(env.laps)
            h["length"].append(env.steps)
            h["crash"].append(int(crashed))
            self.ep_ret = 0.0
            self.obs = env.reset()
        else:
            self.obs = obs2

    def recent(self, key, n=50):
        v = self.hist[key][-n:]
        return float(np.mean(v)) if v else 0.0


# ============================================================================
# 6.  EVALUATION + BASELINE POLICIES
# ============================================================================


def random_policy_factory(seed=0):
    rng = np.random.default_rng(seed)
    return lambda obs: int(rng.integers(CarEnv.N_ACTIONS))


def heuristic_policy(obs):
    """Hand-written baseline: steer towards the side with more free space."""
    r, v = obs[:9], obs[9]
    diff = r[5:].mean() - r[:4].mean()          # > 0  : more room on the left
    if r[4] < 0.25 and v > 0.3:
        return 4                                # brake when something is straight ahead
    if abs(diff) > 0.04:
        return 1 if diff > 0 else 2
    return 3 if v < 0.8 else 0


def evaluate(policy, track_seed=1, episodes=20, max_steps=3000, seed=123):
```

```
    """Run `episodes` rollouts from random start points.  Returns summary metrics."""
    env = CarEnv(Track(track_seed), max_steps=max_steps, rng=np.random.default_rng(seed))
    laps, crashes, speeds = [], [], []
    for _ in range(episodes):
        obs, vsum = env.reset(), 0.0
        while True:
            obs, _, done, _ = env.step(policy(obs))
            vsum += env.v
            if done or env.steps >= max_steps:
                break
        laps.append(env.laps)
        crashes.append(env.crashed)
        speeds.append(vsum / env.steps)
    return {"track": track_seed, "mean_laps": float(np.mean(laps)), "std_laps": float(np.
        std(laps)),
            "survive_rate": float(1.0 - np.mean(crashes)), "mean_speed": float(np.mean(
                speeds)),
            "episodes": episodes, "max_steps": max_steps}


def agent_policy(agent):
    return lambda obs: agent.act(obs, 0.0)


# ============================================================================
# 7.  OPTIONAL FIGURES (matplotlib)
# ============================================================================


def save_figures(trainer: Trainer, out_dir: str, results: dict | None = None):
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except Exception:
        print("matplotlib not installed - skipping figures")
        return
    os.makedirs(out_dir, exist_ok=True)
    h = trainer.hist
    # --- learning curve ---
    if h["step"]:
        fig, ax = plt.subplots(1, 2, figsize=(10, 3.6))
        steps = np.array(h["step"])
        for a, key, ttl in ((ax[0], "laps", "Distance driven per episode (laps)"),
                            (ax[1], "ret", "Return per episode")):
            y = np.array(h[key], dtype=float)
            a.plot(steps, y, color="#b8c4d6", lw=0.6, label="episode")
            k = min(10, max(1, len(y) // 5))
            ma = np.convolve(y, np.ones(k) / k, mode="valid")
            a.plot(steps[k - 1:], ma, color="#1f5fbf", lw=2, label=f"moving avg ({k})")
            a.set_xlabel("training steps"); a.set_title(ttl); a.grid(alpha=.3); a.legend
                ()
        fig.tight_layout(); fig.savefig(os.path.join(out_dir, "learning_curve.png"), dpi
            =160); plt.close(fig)
    # --- trajectory of the trained agent ---
    tr = trainer.env.track
    env = CarEnv(tr, max_steps=1500, rng=np.random.default_rng(5))
    obs = env.reset(start_idx=0, random_start=False)
    xs, ys, vs = [env.x], [env.y], [env.v]
    for _ in range(1500):
        obs, _, done, _ = env.step(trainer.agent.act(obs, 0.0))
        xs.append(env.x); ys.append(env.y); vs.append(env.v)
        if done:
            break
    fig, ax = plt.subplots(figsize=(5.2, 5.2))
    ax.fill(*tr.outer.T, color="#d9dde3"); ax.fill(*tr.inner.T, color="white")
    ax.plot(*np.vstack([tr.outer, tr.outer[:1]]).T, "k", lw=1); ax.plot(*np.vstack([tr.
        inner, tr.inner[:1]]).T, "k", lw=1)
    sc = ax.scatter(xs, ys, c=vs, s=3, cmap="viridis", vmin=1, vmax=5)
    fig.colorbar(sc, ax=ax, fraction=0.046, label="speed")
    ax.set_aspect("equal"); ax.set_title("Greedy trajectory of the trained car"); ax.axis
        ("off")
    fig.tight_layout(); fig.savefig(os.path.join(out_dir, "trajectory.png"), dpi=160);
```

```
        plt.close(fig)
    # --- what the car "sees": the 9 range sensors ---
    env = CarEnv(tr, rng=np.random.default_rng(3))
    env.reset(start_idx=95, random_start=False)
    fig, ax = plt.subplots(figsize=(5.2, 5.2))
    ax.fill(*tr.outer.T, color="#d9dde3"); ax.fill(*tr.inner.T, color="white")
    ax.plot(*np.vstack([tr.outer, tr.outer[:1]]).T, "k", lw=1); ax.plot(*np.vstack([tr.
        inner, tr.inner[:1]]).T, "k", lw=1)
    for d, dist in zip(env.ray_dirs, env.ray_dists):
        ax.plot([env.x, env.x + d[0] * dist], [env.y, env.y + d[1] * dist], color="#
            d9534f", lw=1.2)
        ax.plot(env.x + d[0] * dist, env.y + d[1] * dist, "o", color="#d9534f", ms=3)
    ax.plot(env.x, env.y, "s", color="#1f5fbf", ms=7)
    ax.set_aspect("equal"); ax.set_title("The 9 range sensors (180$^\\circ$ fan)"); ax.
        axis("off")
    fig.tight_layout(); fig.savefig(os.path.join(out_dir, "sensors.png"), dpi=160); plt.
        close(fig)
    # --- comparison bars ---
    if results:
        names = [k for k in results if k.startswith("train_track") or k.startswith("
            unseen")]
        fig, ax = plt.subplots(figsize=(6.4, 3.6))
        w = 0.27
        for j, (pol, col) in enumerate((("random", "#9aa5b5"), ("heuristic", "#e0a030"),
            ("dqn", "#1f5fbf"))):
            vals = [results[n][pol]["mean_laps"] for n in names]
            ax.bar(np.arange(len(names)) + (j - 1) * w, vals, w, label=pol, color=col)
        ax.set_xticks(range(len(names))); ax.set_xticklabels(names, rotation=15)
        ax.set_ylabel("mean laps before crash / time limit"); ax.set_ylim(0, 12)
        ax.legend(ncol=3, loc="upper center"); ax.grid(axis="y", alpha=.3)
        fig.tight_layout(); fig.savefig(os.path.join(out_dir, "comparison.png"), dpi=160)
            ; plt.close(fig)
    print(f"figures written to {out_dir}/")


# ============================================================================
# 8.  TKINTER USER INTERFACE
# ============================================================================


class App:
    """Live view: watch the car learn, change speed, drive it yourself."""

    CAN_W, CAN_H = 820, 700

    def __init__(self, root, trainer: Trainer):
        import tkinter as tk
        from tkinter import filedialog, messagebox
        self.tk, self.fd, self.mb = tk, filedialog, messagebox
        self.root, self.tr = root, trainer
        self.running = False
        self.keys = set()
        self.frame = 0
        root.title("Self-driving car - Deep Q-Learning from scratch (NumPy)")
        root.configure(bg="#14171c")

        self.cv = tk.Canvas(root, width=self.CAN_W, height=self.CAN_H, bg="#1b1f27",
            highlightthickness=0)
        self.cv.grid(row=0, column=0, padx=8, pady=8)
        panel = tk.Frame(root, bg="#14171c")
        panel.grid(row=0, column=1, sticky="n", padx=(0, 10), pady=8)

        self.btn_run = tk.Button(panel, text="Start training", width=22, command=self.
            toggle_run)
        self.btn_run.pack(pady=2)
        for text, cmd in (("Reset car", self.reset_car), ("New random track", self.
            new_track),
                          ("Reset agent (forget)", self.reset_agent),
                          ("Save model...", self.save_model), ("Load model...", self.
                              load_model")):
            tk.Button(panel, text=text, width=22, command=cmd).pack(pady=2)

        self.manual = tk.BooleanVar(value=False)
```

```
        self.greedy = tk.BooleanVar(value=False)
        self.rays = tk.BooleanVar(value=True)
        for text, var in (("Drive manually (arrow keys)", self.manual), ("Greedy: no exploration", self.greedy),
                          ("Show sensors", self.rays)):
            tk.Checkbutton(panel, text=text, variable=var, bg="#14171c", fg="#dfe4ec",
                           selectcolor="#14171c", activebackground="#14171c", anchor="w",
                           command=self._mode_changed).pack(fill="x")
        tk.Label(panel, text="Simulation steps per frame", bg="#14171c", fg="#dfe4ec").pack(pady=(8, 0))
        self.spf = tk.Scale(panel, from_=1, to=400, orient="horizontal", length=190,
                            bg="#14171c", fg="#dfe4ec", highlightthickness=0)
        self.spf.set(8)
        self.spf.pack()

        self.stats = tk.StringVar()
        tk.Label(panel, textvariable=self.stats, justify="left", anchor="w", font=("Courier", 10),
                 bg="#14171c", fg="#9fe0a6").pack(fill="x", pady=8)
        self.plot = tk.Canvas(panel, width=290, height=150, bg="#1b1f27", highlightthickness=0)
        self.plot.pack()
        tk.Label(panel, text="distance per episode (laps), moving avg", bg="#14171c", fg="#8d96a5",
                 font=("Helvetica", 8)).pack()
        tk.Label(panel, text="keys: arrows = drive, space = start/pause", bg="#14171c", fg="#8d96a5",
                 font=("Helvetica", 8)).pack(pady=(6, 0))

        root.bind("<KeyPress>", self._key_down)
        root.bind("<KeyRelease>", self._key_up)
        self._fit_view()
        self._draw_track()
        self._tick()

    # -- coordinate transform -------------------------------------------------
    def _fit_view(self):
        o = self.tr.env.track.outer
        (x0, y0), (x1, y1) = o.min(axis=0), o.max(axis=0)
        self.s = 0.94 * min(self.CAN_W / (x1 - x0), self.CAN_H / (y1 - y0))
        self.cx, self.cy = (x0 + x1) / 2, (y0 + y1) / 2

    def _c(self, x, y):
        return (self.CAN_W / 2 + (x - self.cx) * self.s, self.CAN_H / 2 - (y - self.cy) * self.s)

    def _poly(self, pts):
        out = []
        for x, y in pts:
            out.extend(self._c(x, y))
        return out

    # -- drawing --------------------------------------------------------------
    def _draw_track(self):
        tr = self.tr.env.track
        self.cv.delete("track")
        self.cv.create_polygon(self._poly(tr.outer), fill="#3a404c", outline="#c9d1dd", width=2, tags="track")
        self.cv.create_polygon(self._poly(tr.inner), fill="#1b1f27", outline="#c9d1dd", width=2, tags="track")
        a, b = tr.inner[0], tr.outer[0]
        self.cv.create_line(*self._c(*a), *self._c(*b), fill="#ffffff", width=3, dash=(4, 3), tags="track")

    def _draw_car(self):
        env = self.tr.env
        self.cv.delete("car")
        if self.rays.get():
            for d, dist in zip(env.ray_dirs, env.ray_dists):
                x1, y1 = self._c(env.x, env.y)
                x2, y2 = self._c(env.x + d[0] * dist, env.y + d[1] * dist)
                near = dist / env.L
                col = "#ff5d5d" if near < 0.3 else ("#ffd24d" if near < 0.6 else "#5de08a
```

```
                    ")
675                 self.cv.create_line(x1, y1, x2, y2, fill=col, tags="car")
676                 self.cv.create_oval(x2 - 2, y2 - 2, x2 + 2, y2 + 2, fill=col, outline="",
                        tags="car")
677         c, s = math.cos(env.th), math.sin(env.th)
678         L, W = 9.0, 5.0
679         pts = [(env.x + c * dx - s * dy, env.y + s * dx + c * dy)
680                for dx, dy in ((L, 0), (-L * 0.8, W), (-L * 0.8, -W))]
681         self.cv.create_polygon(self._poly(pts), fill="#4aa3ff" if not self.manual.get()
                else "#ffb347",
682                                outline="white", tags="car")
683
684     def _draw_plot(self):
685         self.plot.delete("all")
686         laps = self.tr.hist["laps"][-400:]
687         if len(laps) < 3:
688             return
689         k = min(15, len(laps))
690         ma = np.convolve(laps, np.ones(k) / k, mode="valid")
691         lo, hi = min(0.0, float(ma.min())), max(0.5, float(ma.max()))
692         pts = []
693         for i, v in enumerate(ma):
694             pts.extend((6 + 278 * i / max(1, len(ma) - 1), 144 - 132 * (v - lo) / (hi -
                    lo)))
695         if len(pts) >= 4:
696             self.plot.create_line(*pts, fill="#4aa3ff", width=2)
697         self.plot.create_text(8, 8, anchor="nw", text=f"{hi:.2f}", fill="#8d96a5", font=(
                "Helvetica", 7))
698
699     def _update_stats(self):
700         t, env = self.tr, self.tr.env
701         self.stats.set(
702             f"steps      {t.total_steps:>9d}\n"
703             f"episodes   {len(t.hist['laps']):>9d}\n"
704             f"epsilon    {t.epsilon():>9.3f}\n"
705             f"avg laps   {t.recent('laps'):>9.2f}\n"
706             f"avg return {t.recent('ret'):>9.1f}\n"
707             f"crash rate {t.recent('crash'):>9.2f}\n"
708             f"td loss    {t.loss_ema:>9.4f}\n"
709             f"speed now  {env.v:>9.2f}\n"
710             f"track seed {t.track_seed:>9d}")
711
712     # -- events ------------------------------------------------------------
713     def _tick(self):
714         if self.manual.get():
715             a = 1 if "Left" in self.keys else 2 if "Right" in self.keys else \
716                 3 if "Up" in self.keys else 4 if "Down" in self.keys else 0
717             env = self.tr.env
718             _, _, done, _ = env.step(a)
719             if done or env.steps >= env.max_steps:
720                 env.reset()
721         elif self.running:
722             greedy = self.greedy.get()
723             for _ in range(int(self.spf.get())):
724                 self.tr.step(greedy=greedy)
725         self._draw_car()
726         self.frame += 1
727         if self.frame % 12 == 0:
728             self._update_stats()
729             self._draw_plot()
730         self.root.after(16, self._tick)
731
732     def _key_down(self, e):
733         if e.keysym == "space":
734             self.toggle_run()
735         else:
736             self.keys.add(e.keysym)
737
738     def _key_up(self, e):
739         self.keys.discard(e.keysym)
740
741     def _mode_changed(self):
742         if not self.manual.get():
```

```
            self.tr.resync()

    def toggle_run(self):
        self.running = not self.running
        self.btn_run.config(text="Pause" if self.running else "Start training")

    def reset_car(self):
        self.tr.obs = self.tr.env.reset()
        self.tr.ep_ret = 0.0

    def new_track(self):
        self.tr.set_track(self.tr.track_seed + 1)
        self._fit_view()
        self._draw_track()

    def reset_agent(self):
        self.tr.reset_agent(seed=int(time.time()) % 10_000)

    def save_model(self):
        p = self.fd.asksaveasfilename(defaultextension=".npz", filetypes=[("NumPy model",
            "*.npz")])
        if p:
            self.tr.agent.save(p)

    def load_model(self):
        p = self.fd.askopenfilename(filetypes=[("NumPy model", "*.npz")])
        if p:
            try:
                self.tr.agent.load(p)
                self.tr.total_steps = max(self.tr.total_steps, self.tr.eps_steps)   # no
                    random exploration
            except Exception as ex:                                                 #
                architecture mismatch etc.
                self.mb.showerror("Could not load model", str(ex))


def launch_gui(model=None, track_seed=1, seed=0):
    try:
        import tkinter as tk
    except Exception as ex:                         # pragma: no cover
        print(f"Tkinter is not available ({ex}).\nInstall it (Linux: sudo apt install
            python3-tk) "
              "or use the headless commands:  train / eval / gradcheck")
        sys.exit(1)
    tr = Trainer(track_seed=track_seed, seed=seed)
    if model:
        tr.agent.load(model)
        tr.total_steps = tr.eps_steps
    root = tk.Tk()
    App(root, tr)
    root.mainloop()


# ============================================================================
# 9.  COMMAND LINE
# ============================================================================


def cmd_train(a):
    os.makedirs(a.out, exist_ok=True)
    tr = Trainer(track_seed=a.track_seed, seed=a.seed, eps_steps=a.eps_steps)
    t0, last = time.time(), time.time()
    while tr.total_steps < a.steps:
        tr.step()
        if time.time() - last > 15:
            last = time.time()
            print(f"step {tr.total_steps:>8d}  eps {tr.epsilon():.2f}  avg laps {tr.
                recent('laps'):6.2f}  "
                  f"crash {tr.recent('crash'):.2f}  loss {tr.loss_ema:.4f}  ({tr.
                      total_steps / (last - t0):.0f} steps/s)")
    secs = time.time() - t0
    print(f"trained {tr.total_steps} steps in {secs:.0f}s ({len(tr.hist['laps'])}
        episodes)")
```

```
    tr.agent.save(os.path.join(a.out, "model.npz"))
    with open(os.path.join(a.out, "history.json"), "w") as f:
        json.dump(tr.hist, f)

    results = {"train_seconds": secs, "steps": tr.total_steps, "episodes": len(tr.hist["laps"])}
    test_tracks = [("train_track", a.track_seed)] + [(f"unseen_{s}", s) for s in a.test_tracks]
    for name, seed in test_tracks:
        results[name] = {
            "random": evaluate(random_policy_factory(7), seed, a.eval_episodes),
            "heuristic": evaluate(heuristic_policy, seed, a.eval_episodes),
            "dqn": evaluate(agent_policy(tr.agent), seed, a.eval_episodes)}
        r = results[name]
        print(f"[{name:>12}] laps  random {r['random']['mean_laps']:6.2f} | heuristic {r['heuristic']['mean_laps']:6.2f}"
              f" | DQN {r['dqn']['mean_laps']:6.2f}   survive(DQN) {r['dqn']['survive_rate']:.2f}")
    with open(os.path.join(a.out, "results.json"), "w") as f:
        json.dump(results, f, indent=2)
    save_figures(tr, os.path.join(a.out, "figures"), results)


def cmd_eval(a):
    ag = DQNAgent()
    ag.load(a.model)
    for seed in [a.track_seed] + list(a.test_tracks):
        r = evaluate(agent_policy(ag), seed, a.eval_episodes)
        print(f"track {seed}: laps {r['mean_laps']:.2f} +- {r['std_laps']:.2f}  survive {r['survive_rate']:.2f}  "
              f"speed {r['mean_speed']:.2f}")


def main(argv=None):
    p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    sub = p.add_subparsers(dest="cmd")
    g = sub.add_parser("gui", help="open the Tkinter interface (default)")
    g.add_argument("--model"); g.add_argument("--track-seed", type=int, default=1); g.add_argument("--seed", type=int, default=0)
    t = sub.add_parser("train", help="headless training + evaluation + figures")
    t.add_argument("--steps", type=int, default=150_000); t.add_argument("--eps-steps", type=int, default=25_000)
    t.add_argument("--out", default="runs/run1"); t.add_argument("--seed", type=int, default=0)
    t.add_argument("--track-seed", type=int, default=1)
    t.add_argument("--test-tracks", type=int, nargs="*", default=[2, 3, 4, 5])
    t.add_argument("--eval-episodes", type=int, default=20)
    e = sub.add_parser("eval", help="evaluate a saved model")
    e.add_argument("--model", required=True); e.add_argument("--track-seed", type=int, default=1)
    e.add_argument("--test-tracks", type=int, nargs="*", default=[2, 3, 4, 5]); e.add_argument("--eval-episodes", type=int, default=20)
    sub.add_parser("gradcheck", help="verify hand-written back-propagation numerically")
    a = p.parse_args(argv)
    if a.cmd in (None, "gui"):
        launch_gui(getattr(a, "model", None), getattr(a, "track_seed", 1), getattr(a, "seed", 0))
    elif a.cmd == "train":
        cmd_train(a)
    elif a.cmd == "eval":
        cmd_eval(a)
    elif a.cmd == "gradcheck":
        sys.exit(0 if gradient_check() < 1e-5 else 1)


if __name__ == "__main__":
    main()
```